Practice 6 — Architectural Design Patterns

Android Development · Project: WeatherPatterns (uz.ttpu.weatherpatterns)

Name: Muslimaxon Odiljonova Group: IT-301-24 Date: 19.10.2026

Part 1 — Why architecture

Question 1 — What a proper architecture gives you

Simplicity (KISS). One class does five different jobs, so to change something small, for example how the temperature is shown in the TextView, I have to scroll through the network code, JSON parsing and SharedPreferences code to find the right place. Nobody can understand the class quickly, and a new person on the team will be afraid to touch it.

Testability. I cannot write a simple unit test for the form validation or the JSON parsing, because they are inside an Activity. To test them I need an emulator (or Robolectric), a real server answer and a real SharedPreferences, which needs a Context. So in practice the logic does not get tested at all.

Low-cost maintenance. Every change touches the same file. If the server changes one JSON field, I edit the same class where the UI is, and I can break the UI by mistake. If two people work on different features they both edit MainActivity.kt and get merge conflicts. Bugs become expensive because everything is connected to everything.

What I would move out first: the server call together with the JSON parsing, into a repository (data layer) behind an interface. It is the biggest and slowest part, it is the part that changes when the backend changes, and it is the hardest to test because it needs a network. After it is behind an interface I can replace it with a fake (like FakeWeatherApi in this practice) and start testing the rest of the class.

Question 2 — Starting from nothing

Plan for my classmate:

1. **Step 1 — Look at the app and its requirements first.** Before choosing any pattern, find out what the app does, which screens are simple and which are complex, where the bugs and the big classes are. The pattern depends on the requirements, so this must be the first step.
2. **Step 2 — Learn the patterns properly** (MVC, MVP, MVVM, MVI): what each layer is responsible for and what its cost is. The lecture says the better you understand a pattern, the better you apply it.
3. **Step 3 — Separate the Model / data layer** (api, model, repository). This is needed by every pattern, so it is useful whatever you choose later.
4. **Step 4 — Choose a pattern per module or screen** and refactor one screen first (the most complex one), not the whole app at once. Simple screens can stay simple.
5. **Step 5 — Add unit tests** for the logic that was moved out, then continue screen by screen. Different modules may end up with different structures, and that is fine.

"Use MVVM everywhere" is not a good answer because there is no single pattern that suits every project. A pattern is abstract and how you implement it depends on the requirements. For a static screen MVVM is just extra classes, and for a screen with a lot of state changing together MVI might fit better. Also, if the classmate copies MVVM without understanding it, he will put the logic in the wrong places and get the same mess, only in more files.

Part 2 — One Model layer for all four patterns

Question 3 — Project and the shared data layer

Project created as Empty Activity (Compose), name WeatherPatterns, package uz.ttpu.weatherpatterns, Min SDK API 24. Packages data.api, data.model, data.repository created, and Weather, WeatherApi and FakeWeatherApi copied in as given.

data/repository/WeatherRepository.kt

```
class WeatherRepository(private val api: WeatherApi) {

    suspend fun getWeather(city: String): Result<Weather> {
        return try {
            val weather = api.fetchWeather(city)
            Result.success(weather)
        } catch (e: IllegalArgumentException) {
            // only the "unknown city" error is turned into a failure,
            // CancellationException is not caught so cancelling still works
            Result.failure(e)
        }
    }
}
```

I used try/catch with IllegalArgumentException only (not runCatching), because runCatching would also catch CancellationException and break coroutine cancellation.

src/test/.../WeatherRepositoryTest.kt

```
class WeatherRepositoryTest {

    private val repository = WeatherRepository(FakeWeatherApi(latencyMs = 0))

    @Test
    fun known_city_returns_success() = runBlocking<Unit> {
        val result = repository.getWeather("Tashkent")

        assertTrue(result.isSuccess)
        assertEquals(24, result.getOrNull()?.temperatureC)
    }

    @Test
    fun unknown_city_returns_failure() = runBlocking<Unit> {
        val result = repository.getWeather("Atlantis")

        assertTrue(result.isFailure)
    }
}
```

Result: both tests pass (green) when run from the gutter arrow, no emulator needed.

Question 4 — Shared screen, module and launcher

I created the package structure from the map (di, ui, mvc/model, mvc/view, mvc/controller, mvp/presenter, mvp/view, mvvm/viewmodel, mvvm/view, mvi/model, mvi/viewmodel, mvi/view). In each view package I created an Empty Activity: MvcActivity, MvpActivity, MvvmActivity, MviActivity. In AndroidManifest.xml there are now four new <activity> entries (android:name=".mvc.view.MvcActivity" etc., with android:exported="false"), next to MainActivity which keeps the LAUNCHER intent filter.

WeatherModule was added to di and WeatherScreen to ui exactly as given, and MainActivity was replaced by the Launcher with the four buttons. When I run the app, each button opens its own (empty for now) screen and Back returns to the launcher.

Part 3 — MVC

Question 5 — MVC roles and the missing Controller

Piece	Role
1. Weather data class	Model
2. WeatherRepository	Model
3. Composable WeatherScreen	View

4. onClick lambda in MainActivity (calls repository, writes TextView)	Controller + View at the same time
5. FakeWeatherApi	Model
6. New class WeatherController	Controller

Piece 4 is a problem because it is the **Controller** (it handles the click and asks the Model for data) and the **View** (it writes the result into the TextView) in one place. Because Android has no separate Controller, this code goes into the Activity, so the Activity keeps growing ("massive Activity"). And the logic can only be tested together with the Activity, which means emulator or instrumented tests, not a fast unit test.

Completed sentence: **the Model has no knowledge of the View** (the user interface).

Question 6 — MVC with a separate Controller

mvc/controller/WeatherController.kt

```
class WeatherController(
    private val model: WeatherModel,
    private val view: WeatherView,
    private val scope: CoroutineScope =
        CoroutineScope(SupervisorJob() + Dispatchers.Main),
) {

    fun onCitySelected(city: String) {
        scope.launch {
            model.startLoading()   // 1. Model goes into loading state
            view.render()          //    and the View redraws ("Loading...")
            model.load(city)       // 2. suspends until the data (or error) arrives
            view.render()          // 3. View reads the Model again
        }
    }
}
```

mvc/view/MvcActivity.kt (render finished, answer in the comment above it)

```
class MvcActivity : ComponentActivity(), WeatherView {

    private val model = WeatherModel(WeatherModule.repository)
    private val controller = WeatherController(model, view = this)
    private var status by mutableStateOf("Pick a city")

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MaterialTheme {
                WeatherScreen(status = status, onCitySelected = controller::onCitySelected)
            }
        }
    }

    // Why no parameters: in MVC the View is allowed to look at the Model itself.
    // The Controller only says "the Model changed", and the View decides what to
    // read from it (isLoading, error, weather).
    // If the Controller passed the Weather in (render(weather)), the View would
    // not need the Model anymore. The Controller would then decide what is shown
    // and we would also need extra methods for loading and error, which is
    // basically MVP (Presenter pushes data into a passive View).
    override fun render() {
        val error = model.error
        val weather = model.weather
        status = when {
            model.isLoading -> "Loading..."
            error != null -> "Error: $error"   // checked before weather, old weather may still
be there
            weather != null -> "${weather.city}: ${weather.temperatureC}°C,
${weather.condition}"
```

```
            else -> "Pick a city"
        }
    }
}
```

I copied error and weather into local vals because they are var properties of another class, so Kotlin cannot smart-cast them. Error is checked before weather because after Tashkent → Atlantis the old weather is still in the Model.

Result when running: the status shows Loading..., then "Tashkent: 24°C, Sunny", "Samarkand: 21°C, Cloudy", "Bukhara: 27°C, Windy", and for Atlantis "Error: No forecast for Atlantis".

Part 4 — MVP

Question 7 — MVP rules: find the violations

(A) breaks rule 1, the View cannot access the Model. The Activity calls repository.getWeather() itself.

(B) breaks rule 3, the View must be dumb and passive. The Activity decides itself what to show ("Sunny" or "Oops") based on the result. This decision belongs to the Presenter.

(C) and (D) do not break the three rules, but the Presenter gets the concrete WeatherActivity and changes its TextView directly. So to test the Presenter I need a real Activity and a real TextView, which cannot be created in a normal JUnit test on the computer; I need an emulator. I also cannot give it a fake View. (Small extra thing: statusLabel is private, so (D) would not even compile.)

The rule the code respects is **rule 2**: there is one Presenter for one View, a one-to-one relationship.

Rewritten constructor (the Presenter only knows an interface):

```
class WeatherPresenter(private val view: WeatherContract.View) {
    fun onRefresh() {
        view.showLoading()   // instead of activity.statusLabel.text = ...
    }
}
```

Question 8 — MVP with a testable Presenter

mvp/presenter/WeatherPresenter.kt (destroy() is added in Question 9)

```
class WeatherPresenter(
    private val view: WeatherContract.View,
    private val repository: WeatherRepository,
    private val scope: CoroutineScope =
        CoroutineScope(SupervisorJob() + Dispatchers.Main),
) {

    fun onCitySelected(city: String) {
        view.showLoading()
        scope.launch {
            repository.getWeather(city)
                .onSuccess { weather -> view.showWeather(weather) }
                .onFailure { e -> view.showError(e.message ?: "Unknown error") }
        }
    }

    // Question 9 fix
    fun destroy() {
        scope.cancel()   // no callback can reach the destroyed View anymore
    }
}
```

src/test/.../WeatherPresenterTest.kt

```
class FakeView : WeatherContract.View {
    val events = mutableListOf<String>()

    override fun showLoading() { events += "loading" }
```

```
    override fun showWeather(weather: Weather) { events += "weather:${weather.city}" }
    override fun showError(message: String) { events += "error:$message" }
}

class WeatherPresenterTest {

    private val view = FakeView()
    private val presenter = WeatherPresenter(
        view = view,
        repository = WeatherRepository(FakeWeatherApi(latencyMs = 0)),
        // Unconfined: the coroutine runs right away, on the test thread
        scope = CoroutineScope(Dispatchers.Unconfined),
    )

    @Test
    fun known_city_shows_loading_then_weather() {
        presenter.onCitySelected("Tashkent")
        assertEquals(listOf("loading", "weather:Tashkent"), view.events)
    }

    @Test
    fun unknown_city_shows_loading_then_error() {
        presenter.onCitySelected("Atlantis")
        assertEquals(listOf("loading", "error:No forecast for Atlantis"), view.events)
    }

    @Test
    fun two_requests_in_a_row_show_two_loading_events() {
        presenter.onCitySelected("Tashkent")
        presenter.onCitySelected("Samarkand")

        assertEquals(2, view.events.count { it == "loading" })
        assertEquals(
            listOf("loading", "weather:Tashkent", "loading", "weather:Samarkand"),
            view.events,
        )
    }
}
```

All three tests pass on the computer. They work without any extra library because the scope is Dispatchers.Unconfined and FakeWeatherApi(latencyMs = 0) does not really suspend, so everything has happened by the time assertEquals runs.

Question 9 — The Presenter that outlives its View

(1)–(2) Before the fix. I ran MvpActivity, tapped Bukhara and rotated within 3 seconds, Logcat filter tag:MVP:

```
D/MVP  onCreate activity=[A]
D/MVP  onDestroy activity=[A]
D/MVP  onCreate activity=[B]
D/MVP  showWeather called on activity=[A]
```

[A] = _____ (first Activity), [B] = _____ (Activity after rotation) ← my numbers from Logcat

showWeather was called on **activity A, the old destroyed one**, not on B. The old Presenter still had its coroutine running and still held a reference to A, so the result went to an Activity that is not on screen anymore (and A cannot be garbage collected until then, so it is a memory leak).

(3) Fix. destroy() cancels the scope and MvpActivity.onDestroy() calls presenter.destroy() (see the code in Question 8 and below). After repeating the experiment the log shows onCreate A, onDestroy A, onCreate B and **no showWeather line at all**: the request is cancelled together with the old Activity, so nothing reaches a destroyed View.

```
// WeatherPresenter
fun destroy() {
    scope.cancel()
```

```
}

// MvpActivity
override fun onDestroy() {
    presenter.destroy()
    Log.d("MVP", "onDestroy activity=${hashCode()}")
    super.onDestroy()
}
```

(4) In both cases the screen after rotation shows **"Pick a city"**. The status is a field of the Activity and the Presenter is created with the Activity, so after rotation a completely new Activity and new Presenter start from the initial value, and the new Presenter never asked for Bukhara. Before the fix the result went to the old Activity, after the fix it is cancelled, so either way the user loses the request. To survive rotation the state (and the running request) has to live in an object that is not destroyed on configuration change, i.e. a **ViewModel** (for small values savedInstanceState could also be used, but it would not keep the request in flight).

Part 5 — MVVM

Question 10 — MVVM: where does each class live?

Package	Classes
data/api	WeatherApi, FakeWeatherApi
data/model	Weather
data/repository	WeatherRepository
presentation/viewmodel	WeatherViewModel, WeatherUiState
presentation/view	WeatherScreen, MvvmActivity
di	WeatherModule

(a) The ViewModel lives longer than the Activity: on rotation the Activity is destroyed and recreated but the ViewModel stays. If it held the Activity or a composable, it would keep the old destroyed one (memory leak, and updates go to the wrong screen, like in Question 9). It would also depend on the Android UI, so it could not be unit-tested. Instead it exposes **observable state**, here a StateFlow<WeatherUiState>, and the View collects it.

(b) Example test: create WeatherViewModel(WeatherRepository(FakeWeatherApi(0))), call onCitySelected("Atlantis") and check that uiState.value == WeatherUiState.Error("No forecast for Atlantis"). This works without any Activity, Compose or emulator, only because the ViewModel does not know the View.

Question 11 — MVVM with a ViewModel and StateFlow

Dependencies added in libs.versions.toml and app/build.gradle.kts:

```
# libs.versions.toml  [libraries]
androidx-lifecycle-viewmodel-compose = { group = "androidx.lifecycle", name =
"lifecycle-viewmodel-compose", version.ref = "lifecycleRuntimeKtx" }
androidx-lifecycle-runtime-compose = { group = "androidx.lifecycle", name =
"lifecycle-runtime-compose", version.ref = "lifecycleRuntimeKtx" }

// app/build.gradle.kts
implementation(libs.androidx.lifecycle.viewmodel.compose)
implementation(libs.androidx.lifecycle.runtime.compose)
```

mvvm/viewmodel/WeatherViewModel.kt

```
class WeatherViewModel(private val repository: WeatherRepository) : ViewModel() {

    private val _uiState = MutableStateFlow<WeatherUiState>(WeatherUiState.Idle)
```

```
    val uiState: StateFlow<WeatherUiState> = _uiState.asStateFlow()

    fun onCitySelected(city: String) {
        _uiState.value = WeatherUiState.Loading
        viewModelScope.launch {
            repository.getWeather(city)
                .onSuccess { weather -> _uiState.value = WeatherUiState.Success(weather) }
                .onFailure { e -> _uiState.value = WeatherUiState.Error(e.message ?: "Unknown
error") }
        }
    }
}
```

di/WeatherModule.kt (final version, MVI factory from Question 13 already included)

```
object WeatherModule {
    // One shared Model layer for all four patterns.
    // 3 seconds of fake latency leaves time to rotate the device in the experiments.
    val repository = WeatherRepository(FakeWeatherApi(latencyMs = 3_000))

    val weatherViewModelFactory: ViewModelProvider.Factory = viewModelFactory {
        initializer { WeatherViewModel(repository) }
    }

    val weatherMviViewModelFactory: ViewModelProvider.Factory = viewModelFactory {
        initializer { WeatherMviViewModel(repository) }
    }
}
```

mvvm/view/MvvmActivity.kt

```
class MvvmActivity : ComponentActivity() {

    private val viewModel: WeatherViewModel by viewModels {
        WeatherModule.weatherViewModelFactory
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MaterialTheme {
                val uiState by viewModel.uiState.collectAsStateWithLifecycle()
                WeatherScreen(
                    status = uiState.toStatusText(),
                    onCitySelected = viewModel::onCitySelected,
                )
            }
        }
    }
}

private fun WeatherUiState.toStatusText(): String = when (this) {
    WeatherUiState.Idle -> "Pick a city"
    WeatherUiState.Loading -> "Loading..."
    is WeatherUiState.Success ->
        "${weather.city}: ${weather.temperatureC}°C, ${weather.condition}"
    is WeatherUiState.Error -> "Error: $message"
}
```

Experiment (tap Bukhara, rotate within 3 s): after the rotation the screen still shows "Loading...", and when the 3 seconds are over it shows "Bukhara: 27°C, Windy". The request was **not lost**. The new Activity gets the same ViewModel instance from the ViewModelStore, viewModelScope is not cancelled on rotation (only in onCleared() when the screen is really finished), and the new Activity just collects the same StateFlow.

Part 6 — MVI

Question 12 — MVI principles: spot the violations

(1) Immutability. The state is several separate public vars that can be changed in place, instead of one immutable state object. One could also say unidirectional flow, because a public var lets anybody write it, but the actual writing from outside happens at (4); the problem at (1) itself is that the state is mutable.

(2) Separation of concerns. The functions are named after widgets (spinner, refresh button), so the ViewModel knows about the View. In MVI the Intent should describe what the user wants (select a city, refresh), not which UI element was touched, and there should be one entry point.

(3) Immutability. It is a data class but with var fields, so a state can be changed after it was created. It should use val and copy().

(4) Unidirectional data flow. The Activity writes the state directly. In MVI data goes only Model → View, and the View talks back only by sending Intents. You can also defend separation of concerns (the View manages state), but I chose unidirectional flow because the main problem is the direction: the arrow goes from the View straight into the state and skips the Intent.

MVI declarations:

```
data class WeatherState(
    val city: String = "Tashkent",
    val isLoading: Boolean = false,
    val temperatureC: Int? = null,
    val error: String? = null,
)

sealed interface WeatherIntent {
    data class SelectCity(val city: String) : WeatherIntent
    data object Refresh : WeatherIntent
}

fun onIntent(intent: WeatherIntent)   // the only public function of the ViewModel
```

Question 13 — MVI: one state, immutable updates, intents

Test dependency added: testImplementation("org.jetbrains.kotlinx:kotlinx-coroutines-test:<version>") with the same version as my coroutines library.

mvi/viewmodel/WeatherMviViewModel.kt

```
class WeatherMviViewModel(private val repository: WeatherRepository) : ViewModel() {

    private val _state = MutableStateFlow(WeatherState())
    val state: StateFlow<WeatherState> = _state.asStateFlow()

    fun onIntent(intent: WeatherIntent) {
        when (intent) {
            is WeatherIntent.SelectCity -> load(intent.city)
            WeatherIntent.DismissError -> {
                _state.update { it.copy(error = null) }
            }
        }
    }

    private fun load(city: String) {
        _state.update { it.copy(isLoading = true, error = null) }
        viewModelScope.launch {
            repository.getWeather(city)
                .onSuccess { weather ->
                    _state.update { it.copy(isLoading = false, weather = weather, error = null)
}
                }
                .onFailure { e ->
                    // old weather is kept on purpose, only the error is added
                    _state.update { it.copy(isLoading = false, error = e.message ?: "Unknown
error") }
                }
```

```
        }
    }
}
```

Factory added to WeatherModule

```
val weatherMviViewModelFactory: ViewModelProvider.Factory = viewModelFactory {
    initializer { WeatherMviViewModel(repository) }
}
```

mvi/view/MviActivity.kt with WeatherState.toStatusText()

```
class MviActivity : ComponentActivity() {

    private val viewModel: WeatherMviViewModel by viewModels {
        WeatherModule.weatherMviViewModelFactory
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MaterialTheme {
                val state by viewModel.state.collectAsStateWithLifecycle()
                WeatherScreen(
                    status = state.toStatusText(),
                    onCitySelected = { city ->
                        viewModel.onIntent(WeatherIntent.SelectCity(city))
                    },
                )
            }
        }
    }
}

// isLoading has to be checked first: while a new city is loading the old
// weather is still inside the state, so if weather was checked first the
// screen would keep showing the old city instead of "Loading...".
private fun WeatherState.toStatusText(): String = when {
    isLoading -> "Loading..."
    error != null -> "Error: $error"
    weather != null -> "${weather.city}: ${weather.temperatureC}°C, ${weather.condition}"
    else -> "Pick a city"
}
```

Which check first: isLoading must be checked first. When I go from Tashkent to Samarkand, the state is isLoading = true but weather is still Tashkent. If weather was checked first the screen would show "Tashkent: 24°C, Sunny" during loading instead of "Loading...".

src/test/.../WeatherMviViewModelTest.kt

```
@OptIn(ExperimentalCoroutinesApi::class)
class WeatherMviViewModelTest {

    @Before
    fun setUp() {
        // viewModelScope runs on Dispatchers.Main, so replace it for the test
        Dispatchers.setMain(UnconfinedTestDispatcher())
    }

    @After
    fun tearDown() {
        Dispatchers.resetMain()
    }

    private fun newViewModel() =
        WeatherMviViewModel(WeatherRepository(FakeWeatherApi(latencyMs = 0)))

    @Test
    fun selecting_a_known_city_ends_with_weather_and_no_loading() {
        val viewModel = newViewModel()
```

```
        viewModel.onIntent(WeatherIntent.SelectCity("Bukhara"))

        val state = viewModel.state.value
        assertFalse(state.isLoading)
        assertEquals("Bukhara", state.weather?.city)
        assertNull(state.error)
    }

    @Test
    fun selecting_an_unknown_city_ends_with_error() {
        val viewModel = newViewModel()

        viewModel.onIntent(WeatherIntent.SelectCity("Atlantis"))

        val state = viewModel.state.value
        assertFalse(state.isLoading)
        assertEquals("No forecast for Atlantis", state.error)
        assertNull(state.weather)
    }

    @Test
    fun dismissing_the_error_clears_it_and_keeps_the_old_weather() {
        val viewModel = newViewModel()
        viewModel.onIntent(WeatherIntent.SelectCity("Tashkent"))
        viewModel.onIntent(WeatherIntent.SelectCity("Atlantis"))
        assertNotNull(viewModel.state.value.error)

        viewModel.onIntent(WeatherIntent.DismissError)

        val state = viewModel.state.value
        assertNull(state.error)
        assertEquals("Tashkent", state.weather?.city)
        assertFalse(state.isLoading)
    }
}
```

All three tests pass. Because every state is just a value, the tests only compare fields of state.value.

Part 7 — Compare and choose

Question 14 — What does each pattern depend on?

Searched with Find in Files (scope: Directory) in mvc, mvp, mvvm and mvi for "import android." and "import androidx.". In every package the android. and androidx. hits are in the Activity (android.os.Bundle, android.util.Log, androidx.activity..., androidx.compose...). The table below is about the class that holds the logic:

Pattern	Logic class	import android.	import androidx.	View holds	Survives rotation
MVC	WeatherController (+ WeatherModel)	No	No	Controller (and the Model)	Nothing, Model and Controller are recreated
MVP	WeatherPresenter	No	No	Presenter	Nothing (request cancelled after fix)
MVVM	WeatherViewModel	No	Yes (lifecycle ViewModel, viewModelScope)	ViewModel	ViewModel: uiState + request in flight
MVI	WeatherMviViewModel	No	Yes (lifecycle ViewModel, viewModelScope)	ViewModel	ViewModel: whole WeatherState + request

MVC: the Controller takes a scope parameter and talks to the View through the WeatherView interface, so it can be tested on the computer with a fake WeatherView and Dispatchers.Unconfined, no extra library.

MVP: the Presenter tests run on the computer with FakeView and Dispatchers.Unconfined, no extra library (done in Question 8).

MVVM: the repository can be tested with runBlocking only, but the ViewModel uses viewModelScope (Dispatchers.Main), so its tests need kotlinx-coroutines-test to call Dispatchers.setMain().

MVI: the same as MVVM, the MVI ViewModel tests need kotlinx-coroutines-test for setMain(UnconfinedTestDispatcher()) (done in Question 13).

Question 15 — Choose a pattern

Situation	Choice	Lecture idea	Cost
(1) About screen, two static texts	No pattern, just a composable (there is not even data to load)	KISS / simplicity; no single pattern suits every project	If the screen later gets dynamic data it has to be refactored
(2) Chat: message list, typing indicator, connection status changing together	MVI	One immutable state + unidirectional data flow keeps all parts consistent; implementation depends on requirements	More boilerplate (state class, intents, copy() on every change) and a harder learning curve
(3) 5-year-old app, 40 Activities reading the DB, 2 devs, release in 1 month	No pattern yet, only a data layer (repositories in front of the database), then patterns screen by screen later	Low-cost maintenance; the pattern depends on the requirements (time, team size)	Activities stay big for now, so testability improves only a little in this release

Can (1) and (2) live in the same app with different structures? Yes. The About screen can be a simple composable and the chat module can use MVI, both using the same data layer. This is supported by the lecture statement that in one complex project **each module can have its own structure**, and that there is no single pattern that suits all Android projects.